Online Resource 6

Re-test census of code-linked speed claims

Supplementary material to: *How the Classical Baseline Shapes Machine-Learning Speed Claims in Topology Optimization.* Shaoliang Yang and Jun Wang, Santa Clara University. Corresponding author: Jun Wang, jwang22@scu.edu. Results in Engineering.

Reading guide. ML/TO, machine learning/topology optimization; CNN, convolutional neural network; CPU/GPU, central/graphics processing unit; OC, optimality criteria; FEM/FEA, finite-element method/analysis; TPMS, triply periodic minimal surface. OAT is Optimize Any Topology, a diffusion model; HGTO is the graph-based design method. L0 means released code ran, including a classical-only run when no learned model was available; L1 compares the article's own implementations here; L2 checks the classical solve/device and evaluated quality. The four comparable outcomes use article-specific criteria: the phase-field CNN compares convergence times and returns lower compliance, while the other three compare a first matching classical stage or solve. The historical frame's P00150 label is corrected to Halle et al. in the dated deviation log; it is not HGTO. Record numbers: P00080, Padhy et al. (VoroTO); P00140, Elingaard et al.; P00150, Halle et al.; P00183, Cang et al.; P00610, Adam et al.; P00647, Viswanath et al.; P00663, Eigel et al. R1–R4 are the audit rules of Online Resource 1; SIMP, solid isotropic material with penalization; top88, the 88-line classical code; TDMA, tridiagonal matrix algorithm; PCG, preconditioned conjugate gradient; AMG, algebraic multigrid; DOF, degree of freedom; FE, finite element; HPC, high-performance computing; MSE, mean squared error; Dice, the Dice overlap coefficient. Article-specific labels (for example NHT) refer to the cited article. In these documents, pre-registered means fixed in writing before the coding or runs it governs, in a hashed plan or a dated log row entered before the run; the hashes are held in the authors' records, without an external registry timestamp.

Reader's note. Evaluative terms in the per-claim results describe our runs of the released code under the census rules, not the merit of the articles or the intent of their authors.

This resource documents the census of every speed claim in the analysis sample whose article links public code (Sect. 3.3.3 and Sect. 4.4 of the paper). The pre-registration was frozen before the first run; every later decision is a dated row of its §9, entered before the run it governs. The pre-registration and the per-claim results are reproduced from the census files, with role labels rendered as 'the authors', internal version labels replaced by plain terms (for example, our GPU baseline), two sentences of internal planning omitted, one text-correction entry shortened, and paths given relative to the census folder of the repository (for example, `runs/P00183/`). Pointers to the paper use its current section numbers. The unedited files and their hashes are in the repository.

Clarification at submission. The historical P00140 plan's wording about an undescribed evaluation refers to the census reconstruction. The article does report fine-scale compliance and volume; the present results below state our inability to reconstruct that comparison. The historical plan and underlying records are retained so the stopping decision remains traceable.

1. Pre-registration and deviation log

Re-test census of code-linked speed claims — pre-registration

Version: 1.0 · **Written:** 2026-09-25 · **Status:** frozen 2026-09-25 in the project manifest, before any timed run; the authors approved the budget rule. Changes go in §9 only.

Why. The main re-tests cover two methods (OAT, HGTO). The census re-tests every speed claim in the analysis sample whose article links public code, so the choice is not ours.

Isolation. Everything for the census lives in its own folder. Files outside it are read, never written. Third-party repositories go in `third_party/<name>@<commit>/` and are not redistributed unless their licence allows it.

---

1. Frame

Speed-stratum records (analysis sample, adjudicated coding) with reported public code availability. The frame is generated by `tools/census_frame.py` into `census/frame.md` and `frame.json`: 7 claims, of which

P00150 (HGTO) was already re-tested. The census re-tests the other 6: P00080, P00140, P00183, P00610, P00647, P00663.

No claim leaves the frame. A claim that cannot be re-run is reported with its outcome level (§3) and the exact blocker.

2. What is re-tested

For each claim, the **primary comparison** is the one the record codes as the claim (primary-outcome paraphrase in `frame.json`). The published factor is the record's claimed speed-up with its stated basis.

3. Outcome levels (each claim reaches the highest level it can)

1. **L0 — runs.** The article's code runs on our machine and produces the article's ML output (or the classical output, when the ML side cannot run).
2. **L1 — same machine.** The article's own ML method and the article's own classical comparator, as published, are timed on our machine under §4. A comparator that the article projected instead of running is run, if its code is public; otherwise the projection is recomputed from our unit costs, and labelled as a projection.
3. **L2 — strong baseline.** The classical comparator's formulation stays fixed (objective, constraints, filter, penalty, update, stopping rule). Only the state solve and the device change, as in `plan/BASELINE_PREREG.md` (purpose paragraph). Time is compared at matched quality (§5).

Reported per claim: published factor; L1 factor; L2 factor at matched quality; the level reached; blockers.

4. Timing and hardware

As `plan/BASELINE_PREREG.md` §4–6: warm-up run discarded, three timed repeats, median with minimum and maximum; GPU timers synchronized; setup reported separately. CPU runs are pinned to P-cores 0–11 (i5-14600KF). The output of the project's `tools/envcapture.py` is copied into `env/`. Published times from other hardware are never mixed with ours in one ratio.

5. Matched quality

The article's own quality evaluation is used on both sides (compliance, effective property, or objective as the article defines it). The ML output's evaluated quality is the target. The classical time is the time to the first iteration that reaches it, with the same thresholding and post-processing on both sides. Where the article compares at a fixed classical iteration count, that count is also reported.

6. Compute budget and failure rule

1. **Training budget: 1 GPU-hour per claim on our RTX 5090** (authors' decision, 2026-09-25). Released weights are used when they exist. Otherwise the model is retrained with the article's code and settings if data generation plus training finishes within 1 GPU-hour of measured wall time on our machine. A run that passes 1 GPU-hour is stopped; the ML side then keeps its published time, labelled "published".
2. **Permitted repairs to reach L0.** Install the stated or nearest available dependency versions; fix file paths and input/output; replace a removed library call with its documented equivalent. Every repair is logged in `runs/<pid>/REPAIRS.md` with the diff.
3. **Blockers.** A claim stops at the level reached, with the blocker named, if going further would need (a) a file that is neither in the repository nor in an archive the article or repository links, or (b) a change to the numerical algorithm or its parameters.
4. **Long classical runs.** A classical run longer than 1 h is timed once, without a warm-up run; its variance is small against its length. All other runs follow §4.

7. Per-claim plan (checked against the repositories at the commits in `third_party/COMMITS.json`)

PID	Primary comparison (record)	Repository contents	ML side	L1 comparator	L2 strong baseline
P00080 VoroTO	0.24 min vs projected 528 min (concurrent homogenization) and 4,986 min (fine-scale TO, 1600×800)	MIT; notebooks for dataset, training and TO; no weights	Retrain (article: 28.4 min data and training on an M2 Air)	Run VoroTO; recompute both projections from our unit costs, labelled as projections	Fine-scale 2D SIMP at 1600×800 with our GPU baseline
P00140 deep-dehom	CNN de-homogenization 1.3–2.2 s vs about 10 s least-squares de-homogenization (cross-paper)	No licence; homogenization inputs in `Output_TO/`; weights and data linked at data.dtu.dk	Linked weights (separate download, needs approval); else retrain	The least-squares comparator is not public: L1 covers the CNN side and top88 at the fine mesh, as the article's alternative comparator	Our GPU baseline at the fine mesh (1440×720 and up)
P00183 Theory-driven TO	about 10^{-2} s generation vs 10^2–10^3 s TO (40×120)	No licence; MATLAB augmented-Lagrangian TO (`infill_topology_optimization.m`), Python learning, training data; no weights	Retrain	Run the repository's MATLAB TO (R2023b installed)	Same augmented-Lagrangian formulation, state solve by sparse direct with reuse or on the GPU
P00610 CNN-Keff	15 ms CNN vs 34.2–371.2 s finite-volume solve (128×128)	CNN-Keff: weights for k_s = 10, 50, 100, 341.7, no licence; Keff-CFD: CPU (C++) and GPU (CUDA) solvers, MIT	Released weights	Keff-CFD CPU solver, as in the article's Table 1	Same finite-volume discretization with a sparse direct or multigrid-preconditioned solver
P00647 DL3DTO	0.001 CPU-h prediction vs 2.4 h per homogenization SIMP run (32^3, one core)	No licence; `Top3D_maxbulk.m`, `Top3D_maxshear.m` (600-iteration cap, stop at change < 0.01), training script; no weights, no data	Published (training 5.5 GPU-h plus 2,751 TO runs exceeds the budget)	Run the repository's MATLAB code, one core	Same formulation; factorization reused across the 6 load cases, or GPU PCG
P00663 DeepNN phase-field	245 s vs 283 s per run, CNN steps inside the same FEniCS loop (10-run averages)	No licence; CNN weights (`.pth`), FEniCS 2019.1 code	Released weights	Both algorithms in Docker (FEniCS 2019.1 image, separate download)	Same loop with a stated efficient linear solver (sparse direct or AMG-preconditioned CG)

Expected ceilings: P00140 reaches L2 only on the top88 path; P00647's ML side stays published.

8. Reporting

All six claims appear in the paper and its Online Resource, whatever their level. The census is summarized as: claims reaching L1 and L2; the published factor against the L2 factor per claim; the blockers.

9. Deviations

Date	Change	Reason
2026-09-25	P00610: classical code at Keff-CFD commit 720be71 (2023-07-22), not HEAD	720be71 is the last commit before the article record (Crossref, 2023-10-19); HEAD (2025) changes the stopping rule. Solver: the TDMA line sweep the article describes (`TDMAscanner`, present in the same file) and the released default (Gauss-Seidel) are both run. See `runs/P00610/REPAIRS.md`
2026-09-25	P00647: L1 and L2 sample fixed before any timing: bulk-modulus runs on rows 438, 1696 and 2082 of `vfrmin.mat` (numpy `default_rng(20260925).choice(2751, 3, replace=False) + 1`); one core; each run timed once (§6.4)	One run takes hours on one core (smoke test: 1 iteration within 7 min including start-up); §7 did not fix the number of runs

Date	Change	Reason
2026-09-25	P00610 timing protocol, fixed before any timed run: 12 images, 3 per folder (numpy `default_rng(20260925)`, folders in the order CirclePack, EllipsePack, QuadrilateralPack, QSGS, from the 100 sampled), 4 ratios. Classical L1: `keff2D_tdma.exe` (article method) and `keff2D_gs.exe` (released default), input defaults, 1 thread, the time the program reports. ML: released CNN, TensorFlow CPU, 1 thread, batch 1, direct model call. L2: same system by SuperLU, 1 thread, time for matrix assembly + factorization + solve + k_eff. All processes pinned to the performance cores (`tools/pin.py`); warm-up + 3 repeats, median. Quality: agreement with the published k_eff on all 100 images × 4 ratios. GPU CNN timing is not possible here (TensorFlow has no GPU support on native Windows); the published 15 ms (RTX 3070) is reported as published	The article compares one CPU core against one CPU core in Sec. 4.2 (20 ms CNN); the dataset images are the authors own
2026-09-25	P00080 protocol, fixed before any timed run. Budget: data generation 736.5 s + training 122.7 s = 14.3 min (within §6.1). ML: `topopt()` of the released `voronoi_TO` notebook (300 epochs, settings.yaml), CPU as released, pinned; warm-up + 3 repeats; the notebook seeds nothing, so each repeat starts from its own random design; time of `topopt()` only; the dehomogenization to a 1600×800 image (`plot_voronoi`, threshold 0.95) timed separately. Quality: final J of each run, and the compliance of the dehomogenized 1600×800 image evaluated at fine scale (E_min 1e-9, our GPU solver). L1: both projections recomputed from our unit costs, with the article formulas (unit cost × 800 macro elements × 300 iterations; one fine-scale iteration × 300): homogenization of one 40×40 micro-image (`homogenize.Homogenization`, as in data generation; median of 20 images, warm-up + 3 repeats) and one iteration of the top88-faithful CPU SIMP (`retest/baseline/simp_cpu.py`, SuperLU) at 1600×800. L2: our GPU SIMP (`retest/baseline/simp_gpu.py`) on the same mid-cantilever (left edge clamped, unit downward load at the right-edge midpoint), 1600×800, volume fraction 0.5, rmin = 1.5·nely/64 = 18.75 (the rule of the baseline pre-registration), 300 iterations; reported: time for 300 iterations and time to the first iteration whose compliance is at most the median fine-scale compliance of the VoroTO designs	The article projects both classical times instead of running them (record P00080)
2026-09-25	P00080 L1 unit iteration (`cpu_iter`): rmin = 1.5 instead of 18.75, decided before that run	At rmin = 18.75 the explicit CPU filter matrix at 1600×800 has about 1.4 billion non-zeros ($\pi \cdot 18.75^2$ per element × 1.28 million elements), which does not fit in memory. The article projection concerns the cost of one fine-scale iteration and states no filter; with rmin = 1.5 the iteration is dominated by the solve. The L2 GPU baseline (filter by convolution) keeps rmin = 18.75
2026-09-25	P00183 protocol, fixed before any timed run. Case 1 (load angle θ over [0, π], 100 test angles `linspace(0, pi, 100)`, 120×40 mesh), the only case whose MATLAB files are all in the repository. ML: the retrained generator (working copy `runs/P00183/work`, repairs in `work/PATCHES.diff`), TensorFlow CPU, 1 thread, forward pass for its fixed batch of 5 angles, time per angle = batch time / 5; warm-up + 3 repeats of 50 calls. Classical L1: the repository augmented-Lagrangian TO, called through `matlab/infill_logged.m` (logging only; diff in `matlab/infill_logged.diff`), `maxNumCompThreads(1)`, pinned, on angles 16, 62 and 75 of the 100 (numpy `default_rng(20260925).choice(100, 3, replace=False) + 1`); warm-up + 3 repeats per angle, or one run if the first exceeds 1 h (§6.4). Quality: the repository evaluator `calculate_compliance.m` (β = 16) on the generator output and on the classical design at the end of each projection stage. Matched quality: classical time to the first stage-end design whose compliance is at most the generator compliance for the same angle; if none, the full run. L2: the same code with MATLAB default threading on the performance cores (the state solve on 6 cores instead of 1); no GPU port	The record gives 10^{-2} s (generator, Xeon E5-1620) against 10^2–10^3 s (TO); the TO algorithm makes about 5,000 FE analyses per run, so its cost is the algorithm, which L2 keeps
2026-09-25	P00183: the released TO function fails (load vector not preallocated); the logged copy preallocates it as the repository evaluator does	Repair to reach L1; see `runs/P00183/REPAIRS.md` 2. The first L1 attempt stopped at this error before any timing
2026-09-25	P00183: the released evaluator `calculate_compliance.m` fails after computing the compliance (undefined `rho_bar`); `compliance_eval.m` returns the compliance only	Repair to score designs; `runs/P00183/REPAIRS.md` 5. The second L1 attempt stopped at this error; its one completed run was the warm-up of angle 16 and is discarded

Date	Change	Reason
2026-09-25	P00647 L2 fixed before any timed run: the same MATLAB code (`runs/P00647/run_rows.m`) with MATLAB default threading on the performance cores (pinned, high priority), rows 438, 1696, 2082, each run timed once; L1 = the same with `maxNumCompThreads(1)`. No GPU port is written (§7 listed it as an option). Runs are launched through `runs/P00647/launch.bat`, one MATLAB session at a time, L1 first	The state solve is MATLAB backslash on the reduced periodic system with 6 right-hand sides (a sparse direct factorization reused across the load cases), already an efficient solver; the remaining lever without new code is the device (1 core → 6 cores)
2026-09-25	P00140 protocol, fixed before any timed run. Weights: figshare 16902961 files downloaded by the authors from the figshare page (zip `16902961.zip`), MD5 equal to the figshare record. ML (L0): `runs/P00140/dehom.py`, the notebook pipeline on the 8 released 60×30 Michell inputs (`Output_TO/michellCantileverSym_*_60_15_*`) with the step-2 weights for periods 10 and 20; CPU, PyTorch default threads, pinned; warm-up + 3 repeats; fidelity = volume fraction V_f and fine-mesh size against Table 2 (article: V_f 0.2695/0.2581 for μ_min 0.05 and 0.4389/0.4323 for μ_min 0.20; t_f 1.28–2.23 s). L1: the least-squares de-homogenization comparator is not public (blocker); the article alternative, top88 at the fine mesh (16 s and 87 s per iteration × 1000 iterations = 16,000 s and 87,000 s), is recomputed with one iteration of the top88-faithful CPU SIMP (`retest/baseline/simp_cpu.py`, SuperLU, rmin 1.5) at 1440×720 and 2400×1200, warm-up + 3 repeats. L2: our GPU SIMP at the same meshes, rmin 1.5, 1000 iterations (the article count), one warm-up at 20 iterations and 3 timed repeats. Matched quality: not attempted (the supports are given only in Fig. 9 of the article and the evaluation of C_f is not described), so no L2 factor at matched quality is reported. Concurrency: these runs overlap the P00647 L1 MATLAB run (one thread); the overlap windows are logged in `runs/P00647/CONCURRENT.log`	Boundary conditions and quality evaluation cannot be taken from the article text; reconstructing them from a figure would add a choice the census cannot justify
2026-09-25	P00663 protocol, fixed before any run. Environment: image `census-p00663:2` (`runs/P00663/env/Dockerfile`; conda-forge Linux builds of the repository versions: FEniCS/mshr 2019.1.0, PETSc 3.14.5, SuperLU_DIST 6.2.0, Python 3.9, PyTorch 1.8.0 CPU), authors' approval 2026-09-25. Runs: `runs/P00663/run663.py`, notebook Experiment 3.1.3 with the nominal load (setting 0; Table 1 and the first row of Table 4: classical 538 iterations, 4 min 43 s; CNN 441 iterations, 4 min 05 s; averages of 10 runs); one container per run, JIT cache on a persistent volume; order: 1 warm-up per algorithm, then 3 timed runs alternating classic/cnn; timed interval = the optimization call, as the notebook brackets it. Quality: final J and compliance reported by the code. Threads and CPUs: Docker Desktop defaults; the container cannot be pinned to the host performance cores, so absolute times are not comparable with the other claims, only the ratio between the two algorithms. L2: the classical loop already solves every system with UMFPACK, a sparse direct solver (efficient under rule R2), so L2 coincides with L1; no solver change is made. Concurrency: overlaps the P00647 L1 MATLAB run (one thread), logged	The article speed-up (1.16) compares two variants of the same code on the same machine; the census checks that ratio
2026-09-25	Concurrency: the P00663 runs overlap the P00140 GPU runs (one host core for kernel launches and PCG control) and the P00647 L1 MATLAB run	The P00140 GPU runs take about 25 min per 1000 iterations at 1440×720 (rmin 1.5 needs many PCG iterations), about 5 h in all; P00663 reports a ratio of two algorithms run alternately in the same container, and extra load can only slow the P00140 GPU baseline, which favours the ML side there. Windows in `runs/P00647/CONCURRENT.log`
2026-09-25	P00663 image `census-p00663:3` = `:2` plus conda-forge `cxx-compiler`, `make`, `pkg-config` (`env/Dockerfile.jit`); the first run attempt (11:24–11:25) stopped because FEniCS JIT found no C++ compiler, and all its outputs were deleted	FEniCS compiles its forms at run time; the macOS export listed its own toolchain, the Linux image needs gcc
2026-09-25	P00663: `run663.py` calls `tic()` before the classical loop, as the notebook does; the second attempt (11:30–11:37) is kept in `runs/P00663/attempt2_discarded/` and rerun in full (its CNN runs completed; its classical runs converged at 538 iterations and then stopped in the code timer `tac()`)	Driver omission, not a code change

Date	Change	Reason
2026-09-25	P00140 GPU at 2400×1200: the timed 1000-iteration run (started about 12:16 in the same process as its warm-up) was stopped at 17:40 with the GPU memory full (32,073 of 32,607 MiB), so its timing is not valid. The three 1440×720 runs completed (1,536, 1,546, 1,566 s; values taken from the run log because the process was stopped before writing its JSON). 2400×1200 will be measured as a unit cost in a fresh process after the CPU unit runs: 100 iterations, no warm-up in that process, median time of iterations 11–100 × 1000 as the projection, with peak GPU memory recorded	Memory oversubscription on Windows falls back to system memory and does not measure the GPU baseline; the article number (87,000 s) is itself a unit-cost projection
2026-09-25	P00140 2400×1200 classical side not measured: the CPU unit run (SuperLU) exited without output while setting up 2400×1200 (17:46; the 1440×720 unit runs completed), and the GPU unit run in a fresh process did not finish 100 iterations within its 30-minute limit (above 18 s per iteration, against 1.55 s at 1440×720, consistent with GPU memory oversubscription). The classical comparison is reported at 1440×720 only	Hardware limits of this machine (95.8 GiB RAM, 32 GB GPU); no further attempt
2026-09-25	P00647 stopped by the authors' decision (19:16): L1 row 438 stopped after 253 iterations without meeting its stopping rule (design change oscillating 0.014–0.026 over iterations 216–251, threshold 0.01; cap 600); rows 1696 and 2082 not run; the full L2 runs replaced by a unit-cost measurement: a copy of Top3D_maxbulk.m with the loop cap set to 10 and a per-iteration timer (`runs/P00647/make_unit.py`), row 438, run once with one thread and once with MATLAB default threads, pinned, sequential, nothing else running; median of iterations 2–10	At about 2.1 min per iteration the pre-registered schedule would end about 2026-09-29; the ML side is published only (no weights or data) and the article scores quality by voxel error only, so no matched-quality comparison is possible and further runs cannot change the claim level. The L1 elapsed time and iteration count are reported as a lower bound on one run
2026-09-25	Correction to §1: P00150 is Halle et al. (2021), not HGTO (HGTO is record P00512, outside the analysis sample). Its re-test was attempted earlier and stopped by the pre-registered reconstruction check (57 of 100 problems within 1 %, 95 required; Sect. 3.3 of the paper and Online Resource 5). The census frame keeps it with that outcome and does not re-run it. `census_frame.py`, `frame.md`, `frame.json` and the README are corrected; the six re-tested claims are unchanged	Labelling error found while writing the census summary; it did not affect which claims were re-tested
2026-09-25	Text correction in the P00140 protocol row: a garbled file path is corrected. No content change	Typing error found on proofreading

2. Claim frame

Record	Year	Published factor	R1	R2	R3	R4	Repository	Status
P00080	2024	2200	partial	partial	pass	partial	github.com/UW-ERSL/VoroTO	to re-test in the census
P00140	2021	5	partial	pass	partial	partial	github.com/elingaard/deep-dehom	to re-test in the census
P00150	2021	259	partial	pass	pass	partial	–	attempted earlier; stopped at the reconstruction check (57 of 100 within 1 %, 95 required)
P00183	2019	10000	fail	partial	partial	pass	github.com/DesignInformaticsLab/Theory_Driven_TO	to re-test in the census
P00610	2023	2280	pass	partial	pass	pass	github.com/adama-wzr/CNN-Keff, github.com/adama-wzr/Keff-CFD	to re-test in the census
P00647	2024	2400	partial	partial	pass	fail	github.com/AshaV17/DL3DTO-Deep-Learning-for-3D-Topology-Optimization	to re-test in the census
P00663	2022	1.16	pass	pass	fail	partial	github.com/MarvinHaa/DeepNNforTopoOptisation	to re-test in the census

3. Results and repairs by claim

P00610 (CNN-Keff) — census result

Level reached: L2. Sources: `summary.json` (from `timing/*.json` and `validate/results.csv`), protocol in Sect. 1, §9. All timed processes pinned to the performance cores, one thread, 2026-09-25.

Claim (record P00610). The CNN predicts k_eff of a 128×128 image in 15 ms (GPU), "3 to 5 orders of magnitude faster" than the finite-volume (NHT) simulation; Table 1 gives 34.2–371.2 s per image per core.

Fidelity check (L0). On 100 of the authors' own images × 4 ratios, the article-era solver reproduces the published k_eff (median absolute relative difference 0.02–0.33 % for TDMA at the released tolerance). A direct solve of the same finite-volume system reproduces them to 1.3e-7–4.3e-6 (median), so the discretization

is identical. The released iterative solvers stop early on some images (up to 20 % off); the published values were computed to tighter convergence than the released default.

k_s	Published factor (Table 1 CPU ÷ 15 ms GPU)	Article solver (TDMA), s	CNN, 1 CPU thread, s	L1 factor	Direct solve (SuperLU), s	L2 factor at matched quality
10	2,280	2.81	0.0088	320	0.0321	3.65
50	5,440	4.23	0.0089	477	0.0320	3.60
100	9,153	5.47	0.0088	622	0.0318	3.61
341.7	24,747	9.82	0.0088	1,112	0.0318	3.59

Medians over 12 images of the median of 3 repeats. The released Gauss–Seidel solver gives L1 factors of 211–797.

Quality. Median error against the published k_eff: CNN 0.14–0.91 % (95th percentile up to 2.35 %); direct solve below 1e-5. The direct solve is at least as accurate as the CNN at every ratio, so the L2 factor is at matched (or better) quality.

Reading. Most of the published factor comes from the classical side: the same released solver runs 12–38 times faster here than Table 1 reports (hardware and tolerance differ; the article names no CPU for Table 1), and an iterative line solver is replaced by a direct solve. Against an efficient solve of the same equations, the CNN keeps a 3.6-fold per-image advantage on one core, before counting its 520,000 training simulations and training.

Not measured. The CNN on a GPU (TensorFlow has no GPU support on native Windows); the published 15 ms is kept as published. A 1-thread CPU CNN (8.8 ms) is faster than the published GPU time, so this does not favour the classical side.

P00610 — repairs and choices (census pre-registration §6.2)

1. **Code version.** Keff-CFD commit `720be71` (2023-07-22), checked out as `third_party/Keff-CFD@720be71`. It is the last commit before the article's Crossref record was created (2023-10-19); the 2D solver has no later change before 2023-11-09. HEAD (`48700a8`, 2025) is a later version with a residual-based stopping rule. Logged as a deviation in Sect. 1, §9 (COMMITS.json lists HEAD).

2. **Compiler.** The code targets GCC. With MSVC 2022 (`/openmp:llvm`), automatic loop scheduling is rejected. The build copy `build/src/` replaces automatic loop scheduling with static loop scheduling in 10 setup loops of `keff2D.cpp` (cell centres and array initialisation). Loop scheduling does not change results. `/FIstring` supplies the `<string>` header that GCC includes implicitly.

3. **Solver variant.** At `720be71` the single-image path calls the released Gauss–Seidel solver. The article describes an iterative TDMA line sweep. `build/src_tdma/` switches the single-image call to the TDMA line-sweep function that the same file defines (the call is present, commented out). Both executables are kept: `keff2D_gs.exe` (code as released) and `keff2D_tdma.exe` (solver the article describes).

Smoke test (2026-09-25, `smoke/`, image `00009.jpg` from the repository, k_s = 10, one core, not pinned, not a timed run): Gauss–Seidel 0.99 s, 12,800 iterations, keff 5.2974; TDMA 1.49 s, 7,300 iterations, keff 5.2990. Article Table 1 for k_s = 10: 34.2 s and 32,903 iterations per image (the images differ).

1. **Image input.** 75 of the 100 sampled dataset JPEGs (CirclePack, EllipsePack, QuadrilateralPack) are stored as 3-channel RGB with identical channels (maximum channel difference 0); the solver rejects images with more than one channel. They are converted to 8-bit grayscale (exact for equal channels) and saved as PNG, so no second JPEG compression occurs. QSGS images are already one-channel. Same conversion for the CNN input.

P00080 (VoroTO) — census result

Level reached: L2. Sources: `timing/{voroto,homog,cpu_iter,gpu,posthoc_threshold}.json`; protocol: Sect. 1, §9 (P00080 rows). All runs pinned to the performance cores, 2026-09-25.

Claim (record P00080). On the 40×20 mid-cantilever, VoroTO optimizes in 0.24 min after a one-time 28.4 min of data generation and training, against a projected 528 min for concurrent homogenization-based

multiscale TO (2,200×) and a projected 4,986 min for brute-force fine-scale TO at 1600×800 (20,775×). Both classical times are projections from unit costs × 300 iterations; neither was run.

ML side (L0). The surrogate was retrained with the released notebooks (repair: `REPAIRS.md`): data generation 736.5 s and training 122.7 s, 14.3 min in total (article: 28.4 min). VoroTO's optimization call: median 11.48 s (3 repeats, 11.38–11.49 s; article 0.24 min). Dehomogenization to a 1600×800 image: median 20.5 s more. Fine-scale compliance of the thresholded images: median 88.98 at volume fraction 0.552–0.559 (homogenized J 78.74 at 0.51).

Comparator	**Article**	**Ours**	**Factor vs VoroTO 11.48 s**
Concurrent homogenization, projected (unit × 800 × 300)	0.132 s unit → 528 min	0.0369 s unit (40×40 micro-mesh, as in data generation) → 147 min	771 (projection)
Brute-force fine-scale TO, projected (one iteration × 300)	16.62 min/iteration → 4,986 min	90.5 s/iteration (top88-faithful CPU, SuperLU, rmin 1.5) → 453 min	2,367 (projection)
L2: our GPU SIMP at 1600×800, volume fraction 0.5, rmin 18.75, to VoroTO's fine-scale compliance (pre-registered)	not run	2.45 s (iteration 9)	**0.21** (the classical run is 4.7× faster)
Our GPU SIMP, full 300 iterations	—	67.7 s, compliance 67.2 (penalized)	5.9

Post hoc check (not pre-registered). The pre-registered rule compares SIMP's penalized compliance with VoroTO's thresholded one. Scoring the SIMP designs the same way (threshold 0.5, fine-scale evaluator): after 9 iterations (2.2 s) 79.1 at volume fraction 0.477; after 20 iterations (5.0 s) 65.7 at 0.495; after 300 iterations 63.0 at 0.501. Each is stiffer than VoroTO's 89.0 while using less material, so the match holds under the stricter scoring.

Reading. Run instead of projected, the classical comparators take 3.6× (homogenization) and 11× (fine-scale) less time than the article's projections on this machine. An efficient fine-scale SIMP reaches VoroTO's structural quality in about 2.5 s, before VoroTO finishes its own optimization and without its 14.3 min offline cost; after 67.7 s its thresholded design has 29 % lower compliance (63.0 against 89.0) with less material. VoroTO's designs are Voronoi multiscale structures, a design space that single-scale SIMP does not produce; the comparison is on the structural objective the article reports.

P00080 — repairs and choices (census pre-registration §6.2)

1. **Sparse-solve compatibility.** VoroTO's `solver.py` imports a sparse linear-solve function from a C++ extension around the SuiteSparse KLU sparse LU solver that does not build on Windows without SuiteSparse. `shim/torch_sparse_solve.py` provides the same call: a sparse direct LU solve (SciPy SuperLU) with the adjoint backward pass. Checked against dense PyTorch autograd on a random 6×6 system: solution, b-gradient and A-gradient agree to below 2e-17. Both are exact direct solvers; the macro problem (40×20 elements) is small, so the solver swap has no material effect on time.

2. **Notebooks.** The dataset, training and optimization code is in three notebooks. `run_notebook.py` runs their code cells unchanged, in order, from a working copy of the repository files (`work/`); plots are not displayed.

3. **Budget clock.** Data generation plus training count against the 1 GPU-hour budget (§6.1) as wall time. VoroTO trains on the CPU (the notebooks never move data to a GPU).

P00183 (theory-driven one-shot generator) — census result

Level reached: L2 (the pre-registered L2 gives no gain here). Sources: `summary.json` (from `timing/gen.json`, `timing/classical_{L1,L2}.mat`), `train.log`, `REPAIRS.md`; protocol: Sect. 1, §9 (P00183 rows). All runs pinned to the performance cores, 2026-09-25.

Claim (record P00183). The trained generator produces a topology in about 10^{-2} s (Xeon E5-1620), against 10^2–10^3 s for the augmented-Lagrangian TO on the 120×40 mesh; about 10^4 faster, data acquisition excluded.

Reproduction (L0). Two released MATLAB files fail as published and were repaired without changing the algorithm: the TO function never sizes its load vector, and the evaluator stops at an undefined variable after computing the compliance (`REPAIRS.md` 2 and 5). The Python/TensorFlow 1 training code was ported with logged changes (`work/PATCHES.diff`). Retraining took 22.4 min (5 active-learning rounds, 9 training designs from the repository's pre-solved set, final test error 0.944 against the stopping criterion 1).

Angle	TO, 1 thread (L1), s	FE analyses	TO compliance	Generator compliance	TO time to match the generator, s	Factor, full TO	Factor, matched quality
16	115.1	4,741	166.4	310.8	57.8	33,615	16,869
62	110.7	4,635	34.2	67.0	56.2	32,331	16,412
75	115.8	4,711	107.6	198.7	57.1	33,829	16,666

Generator: 3.42 ms per angle (one thread, batch of 5). Times are medians of 3 repeats (spread under 2 %). Compliance from the repository evaluator (projection $\beta = 16$). "Time to match" is the first end of a projection stage at which the TO design's compliance is at most the generator's; the first stage end (after about 56–58 s) already does so on every angle.

L2. MATLAB default threading on the 6 performance cores is slower for this 9,922-DOF problem: 135.5–140.3 s per run (matched quality 68.1–69.0 s). The TO cost is its about 4,700 FE analyses of a small system, which the census keeps fixed; a faster solver for the same system was not tested beyond this threading change.

Reading. The claim's order of magnitude holds on our machine: 10^4 at full TO time and about 1.6×10^4 at matched quality. The generator's designs have 1.85–1.96 times the TO compliance on these angles; the TO reaches a better design before its second projection stage. Offline cost (derived): 22.4 min of training plus the 9 pre-solved training designs (about 9×113 s = 17 min of TO) against a saving of about 57 s per design at matched quality, so the generator repays its offline cost after about 41 designs.

P00183 — repairs and choices (census pre-registration §6.2)

1. **Python 3 / TensorFlow 2.** `port.py` makes the working copy `work/` of "CAD code/" at commit 02de10e and patches `main_1D.py` only (`work/PATCHES.diff`): `tensorflow.compat.v1` with v2 behaviour disabled; Python 2 integer division in tensor shapes and loop counts written as floor division; seeds 20260925 for random, numpy and TensorFlow (the original draws its initial training set without a seed); the MATLAB engine is pointed at the working folder before the compliance function for the tested case runs.

2. **Load vector.** The released `infill_topology_optimization.m` never preallocates the load vector F; assigning the two load entries leaves it 9,882 long, and indexing F at the free degrees of freedom fails for the 9,922-DOF model ("Index exceeds the number of array elements", first L1 attempt, 2026-09-25 09:15). The logged copy (`matlab/infill_logged.m`, made by `make_logged.py`) preallocates a sparse load vector with $2(n_y + 1)(n_x + 1)$ entries, where n_x and n_y count grid elements, as the repository's own `calculate_compliance.m` does. The released TO function therefore does not run as published; the 100 ground-truth designs in `experiment_data/phi_true_1D.mat` were produced by code not in the repository (the called ground-truth generation routine is absent).

3. **Case.** Case 2 (`main_adaptiv.py`) calls the second case's compliance function, which is not in the repository; case 1 is used.

4. **Training.** Budget clock 08:52:33–09:14:55 (22.4 min, within §6.1): 5 active-learning rounds, 9 training designs, final test error 0.944 against the stopping criterion 1 (`train.log`). Checkpoint `model_1D_sample_9`.

5. **Evaluator.** The released `calculate_compliance.m` computes the compliance and then stops at an undefined density variable in its constraint lines (second L1 attempt, 09:19, after its first TO run finished). The copy `matlab/compliance_eval.m` (made by `make_logged.py`; diff `matlab/compliance_eval.diff`) returns the compliance and drops the two constraint lines. The compliance computation is unchanged.

P00663 (CNN gradient steps in phase-field TO) — census result

Level reached: L2 = L1. Sources: `summary.json`, `results/*.json`; protocol: Sect. 1, §9 (P00663 rows). Runs in Docker (`census-p00663:3`), 2026-09-25; not pinned to the performance cores (see §9).

Claim (record P00663). On the deterministic 2D bridge under the nominal load, inserting CNN-predicted gradient steps every 55 iterations cut convergence from 538 to 441 iterations, with lower compliance (1148.66 against 1173.22), in 4 min 05 s against 4 min 43 s (averages of 10 runs): 1.16× (derived).

Reproduction (L0). The FEniCS 2019.1 code and the released CNN weights run unchanged in a Linux environment built from the repository's package versions (`env/`). Both runs reproduce the article's iteration counts exactly and its compliances within 0.01: classical 538 iterations, compliance 1173.23 (article 1173.22); CNN-assisted 441 iterations, compliance 1148.66 (article 1148.66).

	Article (Table 1, Table 4)	**Ours (median of 3; repeats)**
Classical loop (Algorithm A1)	538 iterations, 283 s	538 iterations, 48.97 s (48.87, 49.39, 48.97)
CNN-assisted loop (Algorithm 1)	441 iterations, 245 s	441 iterations, 42.76 s (42.76, 43.25, 42.42)
Ratio	1.155	**1.145**

L2. The classical loop solves every system with UMFPACK, a sparse direct solver, which rule R2 counts as efficient; no stronger solver is substituted, so L2 equals L1.

Reading. The claim holds: the CNN steps save 18 % of the iterations and 13 % of the time, and the design has 2 % lower compliance. The absolute times are 5.8 and 5.7 times shorter than the article's on this machine; the comparison is between two variants of the same code on the same machine, so the ratio carries over. Data generation and training are not reported by the article and are not counted here.

P00140 (CNN de-homogenization) — census result

Level reached: L0; L1 and L2 partial (time only, 1440×720 only). Sources: `timing/dehom.json`, `timing/classical_1440.json`; protocol: Sect. 1, §9 (P00140 rows).

Claim (record P00140). CNN de-homogenization of a 60×30 homogenization result takes about 1.3 s (to 1440×720) and 2.2 s (to 2400×1200) on one Intel Core i7 CPU, against about 10 s for least-squares de-homogenization (a cross-paper figure, 80×40 to 1600×800): 5–10× (derived). The article also projects top88 at the fine meshes: 16 s and 87 s per iteration × about 1,000 iterations = 16,000 s and 87,000 s.

Reproduction (L0). The released pipeline with the released weights (repairs: `REPAIRS.md`) on the 8 released 60×30 Michell inputs produces 2400×1200 designs in 1.69–1.94 s (period 10) and 2.05–2.10 s (period 20), medians of 3 (article: 2.16–2.23 s at 2400×1200). Volume fractions match Table 2 within 0.006 (period 20: 0.2693 against 0.2695 for μ_min 0.05, 0.4375 against 0.4389 for μ_min 0.20).

L1. The least-squares de-homogenization is not public; its 10 s is a figure from another paper and is not re-run. One iteration of our top88-faithful CPU port (SuperLU) at 1440×720 takes 75.9 s (median of 3), slower than the article's 16 s for top88, so it does not reduce the article's projection.

L2 (time only). Our GPU SIMP runs 1,000 iterations at 1440×720 in 1,546 s (median of 3; 1,536–1,566 s), against the article's projected 16,000 s. At 2400×1200 neither classical measurement fits this machine (the SuperLU run exited during setup, memory suspected but not confirmed; the GPU run exceeded 18 s per iteration, consistent with memory above 32 GB), so the 87,000 s projection is not re-checked.

Matched quality: not reached. The article reports fine-scale compliance and volume evaluations, but this census did not reconstruct the exact supports and fine-scale compliance evaluation from the released material. The primary least-squares comparator was not public.

Reading. The article's headline factor (5–10× against least-squares de-homogenization) cannot be re-tested because that comparator is not public. The CNN pipeline reproduces its reported times and volume fractions. Against fine-scale SIMP, an efficient GPU run cuts the article's top88 projection at 1440×720 by about 10×, but the CNN (1.28 s at 1440×720 in the article; 1.7–2.1 s at 2400×1200 here) remains about three orders of magnitude faster in time; whether the designs are of comparable quality is not measured. The coarse homogenization-based optimization that feeds the CNN is not public and is not counted on either side.

P00140 — repairs and choices (census pre-registration §6.2)

1. **Weights.** figshare answers scripted downloads with a bot check (HTTP 202, empty body), which the census does not try to get around. The authors downloaded the article's zip (`16902961.zip`, 8,699,051 bytes, four files); the extracted files match the figshare record's MD5 sums (`weights/PROVENANCE.json`).
2. **Stale notebook names.** `lam_width_projection.ipynb` imports the radial-basis-function upsampler and dot-product loss from modules that no longer define them. `dehom.py` uses the released dehomogenization network (same layers; the released weights load with matching keys and shapes) and the dot-product loss from `deepdehom/main.py`.
3. **Encoder folder.** The training-data loader refuses an empty folder but uses the data only for training; the encoder is built on a folder with one placeholder file. The notebook's hard-coded path to the authors' training data is therefore not needed.
4. **scikit-image keyword.** `selem=` became `footprint=` (1 occurrence, `work/PATCHES.diff`, made by `port.py`); the pinned scikit-image 0.18.3 has no wheels for Python 3.12.
5. **Display only.** Verbose output and plotting are disabled in `lam.forward`; they print and plot.
6. **Seed.** The encoder adds Gaussian noise of 1° (notebook setting); numpy is seeded (20260925) before each run.

P00647 (DL3DTO, 3D TPMS unit cell) — census result

Level reached: L1 partial, L2 as a projection; matched quality not possible. Sources: `summary.json`, `run_L1.log`, `unit_1thread.log`, `unit_default.log`; protocol: Sect. 1, §9 (P00647 rows).

Claim (record P00647). Once trained, the 3D ResUNet predicts an optimized 32^3 gyroid unit cell in 0.001 CPU-hours, against about 2.4 h per homogenization-based SIMP run on one HPC core: 2,400× (derived). Quality is judged only by voxel MSE and Dice (rule R4 fail).

ML side. No trained weights and no data are released; training took 5.5 GPU-hours plus 2,751 TO runs, beyond the 1 GPU-hour budget. The prediction time stays as published (3.6 s).

Classical side (L0, L1). The repository's MATLAB code (`Top3D_maxbulk.m`) runs unchanged. On row 438 of the released parameter table, one core, it ran from 10:22:08 to 19:16:43 (32,075 s) and was stopped after 253 iterations by the authors' decision; it had not met its stopping rule (design change below 0.01): the change oscillated between 0.014 and 0.026 over iterations 216–253, and the loop cap is 600. Rows 1696 and 2082 were not run. The long run overlapped other jobs; a clean 10-iteration run of the same row gives a median of 120.0 s per iteration (iterations 2–10), close to the long run's average of 126.8 s.

L2 (projection). The same code with MATLAB default threading (6 threads) follows the identical trajectory (same objective values) at a median of 36.5 s per iteration, 3.3× faster than one thread. Over the 253 iterations that reached, that is about 9,245 s (2.6 h); to the 600-iteration cap, about 21,900 s (6.1 h).

	Article	**Ours**
Classical run, one core	2.4 h	$\geq$ 8.9 h (253 iterations, not converged); about 20 h to the cap (projection, 600 × 120.0 s)
Classical run, 6 threads	—	about 2.6 h for the same 253 iterations (projection)
ML prediction	0.001 CPU-h (3.6 s)	not reproducible (no weights)
Factor	2,400	$\geq$ 8,900 on one core; about 2,570 with 6 threads over 253 iterations (projections, against the published 3.6 s)

Reading. We found no evidence in this run that the published classical time was overstated: on this machine a run of the released code takes longer than the article's 2.4 h, and threading brings it back to about the article's figure. The claim cannot be tested at matched quality: the authors release no model, and they score predictions by voxel agreement, not by the bulk or shear modulus the optimization maximizes. The released code also differs from the article's text (no Heaviside continuation, a 600-iteration cap; record P00647), which may explain the longer run.

4. Third-party code

The table records the downloaded repository snapshots. P00610 was timed with Keff-CFD at the article-era commit 720be71, rather than snapshot 48700a8; Sect. 1, §9 records this choice.

Record	**Repository**	**Snapshot commit**	**Licence**
P00080	https://github.com/UW-ERSL/VoroTO	d407d15d9c87	MIT
P00140	https://github.com/elingaard/deep-dehom	73529a934028	none (no licence file; local use only, not redistributed)
P00183	https://github.com/DesignInformaticsLab/Theory_Driven_TO	02de10e0a04c	none (no licence file; local use only, not redistributed)
P00610	https://github.com/adama-wzr/Keff-CFD	48700a842afa	MIT
P00610	https://github.com/adama-wzr/CNN-Keff	46ec2b655356	none (no licence file; local use only, not redistributed)
P00647	https://github.com/AshaV17/DL3DTO-Deep-Learning-for-3D-Topology-Optimization	d9db8b2dc285	none (no licence file; local use only, not redistributed)
P00663	https://github.com/MarvinHaa/DeepNNforTopoOptisation	6c042e57b289	none (no licence file; local use only, not redistributed)

5. External software identifiers

The repair descriptions use plain terms for software operations. The table gives the exact identifiers in the third-party code for readers reproducing those repairs; the source files and patch differences are located in Sect. 3.

Identifier in external code	Operation in the repair description
`schedule(auto)`	Automatic loop scheduling in Keff-CFD.
`schedule(static)`	Static loop scheduling in the MSVC-compatible copy.
`ParallelGS`	Released Gauss–Seidel solver.
`TDMAscanner`	TDMA line-sweep function in the same source file.
`topopt()`	VoroTO optimization call.
`torch_sparse_solve.solve`	Sparse linear-solve function imported by VoroTO.
`RBF_Upsampler`	Radial-basis-function upsampler named by the stale notebook.
`DotLoss`	Dot-product loss.
`DehomNet`	Released dehomogenization network with matching weight keys and shapes.
`HomogenizationData`	Training-data loader.
`cal_c_1D`	Compliance function for the tested case in the MATLAB working folder.
`infill_1D`	Called ground-truth generation routine absent from the release.
`cal_c_high_dim1_3d`	Compliance function for the unused second case, absent from the release.
`rho_bar`	Undefined density variable in the released constraint lines.
`verbose`, `enable_plot`	Print and plot options in the forward call.